RAG-Enabled FOMC Policy Analysis: Demonstrating Grounded Monetary Policy Q&A with Lightweight Open-Source Models

Table of Contents

1. Introduction & Problem Statement 1
   1.1 Why RAG? 2
2. Data Sourcing & Processing 2
   2.1 Data Source 2
   2.2 Text Cleaning 3
   2.3 Chunking Strategy 4
3. Methodology & Implementation 8
   3.1 Pipeline Architecture 8
   3.2 Embedding & Indexing 8
   3.3 Hybrid Retrieval 9
   3.4 Generation 13
4. Evaluation & Results 15
   4.1 Evaluation Framework 15
   4.2 RAG vs Base Model Comparison 19
   4.4 Retrieval Ablation Study 21
   4.5 Chunk Size Comparison 21
   4.6 Embedding Model Comparison 21
   4.7 Sample Output 22
5. Discussion & Critical Analysis 23
   5.1 Where the System Succeeds 23
   5.2 Where the System Struggles 23
   5.3 Challenges Encountered 24
   5.4 Future Work 24
6. Conclusion 25

AI Tool Declaration

Claude (Anthropic, Claude Opus 4.6) was used to assist with brainstorming the problem statement, structuring RAG pipeline, writing and debugging code, and drafting sections of this report. Claude Haiku 4.5 was used via API as an LLM judge for evaluation. I am responsible for the content and quality of the submitted work.

1. Introduction & Problem Statement

Financial analysts, journalists, and researchers frequently need to understand the Federal Reserve's position on specific economic topics — inflation, employment, interest rates, quantitative tightening — at particular points in time. Currently, this requires manually reading through lengthy FOMC documents where minutes alone can exceed 20 pages per meeting. With eight scheduled meetings per year and multiple document types per meeting, staying up to date with Fed policy requires substantial reading effort.

This project builds a **RAG-based Q&A system** that, addresses the following problem statement:
Given a natural-language query about Fed monetary policy, retrieves relevant passages from FOMC documents and generate a grounded, context-aware answer.

The system targets FOMC meeting minutes and statements from 2020 to 2025, covering the COVID-19 pandemic response, the historic tightening cycle, and the subsequent rate-cutting period.

1.1 Why RAG?

A key question is whether this problem truly requires retrieval-augmented generation. The evaluation directly addresses by providing a RAG pipeline, then comparing the chosen model, Mistral 7B's answers with and without retrieval context on identical queries. In doing so, we observe that base model demonstrates some familiarity with FOMC policy — identifying correct meeting dates, general policy direction, and committee structure for major events. This suggests that publicly available Federal Reserve documents may be represented in its training corpus.

The results demonstrate that while the base model can produce plausible-sounding answers about well-known meetings, it frequently hallucinates specific details e.g. inventing meeting dates that don't exist (e.g., "February 1 and March 21, 2024" when the actual dates were January 31 and March 20), fabricating statistics ("unemployment rate declining to 3.5%"), and confidently presenting unverifiable claims. RAG addresses this by grounding every claim in an actual source document, providing an audit trail essential in financial contexts where accuracy and traceability are non-negotiable.

Furthermore, this RAG pipeline demonstrates how a lightweight 7-billion parameter model to produce accurate, grounded policy analysis, without API costs or proprietary dependencies. Such a RAG pipeline could potentially help democratize access to quality financial NLP by reducing the compute requirements for reliable outputs.

2. Data Sourcing & Processing

2.1 Data Source

vtasca/fomc-statements-minutes dataset from HuggingFace was chosen as a database. It contains FOMC meeting statements and minutes from 2000 to December 2025 and provides four fields per document: meeting date, release date, document type (Statement or Minute), and the full text content. We filtered to the 2020–2025 period, yielding approximately 96 documents (48 meetings × 2 document types).

This time range was chosen as it covers an interesting policy period riddled with COVID responses, historic tightening cycle and rate cuts. Focusing on these recent 5 years also allowed us to focus on more consistent language styles of this period.

2.2 Text Cleaning

FOMC documents contain substantial boilerplate that would disrupt retrieval. Our cleaning pipeline addresses several issues:

- **Encoding artifacts:** The dataset contains corrupted Unicode characters (e.g., 'â' appearing where em dashes should be) most likely caused by UTF-8 text being misinterpreted as Latin-1. We implemented a fix_encoding() function that attempts a Latin-1 to UTF-8 re-encode, with fallback manual replacements for common patterns.
- **Boilerplate removal:** Removing attendance lists, notation vote sections, footnotes, implementation note references, and media inquiry lines from minutes. However, section headers (e.g., "Staff Review of the Economic Situation") were preserved as they provide useful topical context for embedding and retrieval.

The following code snippet shows the `fix_encoding()` and `clean_text()` functions used in text cleaning.

```
def fix_encoding(text):
    """
    Fix common encoding artifacts in FOMC documents.
    These occur when UTF-8 text is misinterpreted as Latin-1/Windows-1252.
    """
    if not isinstance(text, str):
        return ""

    # Try to fix mojibake by re-encoding
    try:
        fixed = text.encode('latin-1').decode('utf-8')
        return fixed
    except (UnicodeDecodeError, UnicodeEncodeError):
        pass

    # Manual replacements for common patterns
    replacements = {
        '\u00e2\u0080\u0093': '–',
        '\u00e2\u0080\u0094': '—',
        '\u00e2\u0080\u0099': "'",
        '\u00e2\u0080\u009c': '"',
        '\u00e2\u0080\u009d': '"',
        '\u00e2\u0080\u0098': "'",
        '\u00c2\u00a0': ' ',
        'â\x80\x93': '–',
        'â\x80\x94': '—',
        'â\x80\x99': "'",
        'â\x80\x9c': '"',
        'â\x80\x9d': '"',
        'â': '—',
    }
```

```
    for bad, good in replacements.items():
        text = text.replace(bad, good)

    return text

def clean_text(text):
    """
    Clean raw FOMC document text.
    Fixes encoding, removes boilerplate, preserves section headers.
    """
    if not isinstance(text, str):
        return ""

    text = fix_encoding(text)
    text = re.sub(r'(?<=[a-zA-Z])\d+(?=\s)', '', text)

    attendance_patterns = [
        r'\nAttendance\s*\n.*$',
        r'\nNotation Vote\s*\n.*$',
        r'\n_+\s*\n.*$',
    ]
    for pattern in attendance_patterns:
        text = re.sub(pattern, '', text, flags=re.DOTALL)

    text = re.sub(r'Implementation Note issued.*$', '', text,
flags=re.MULTILINE)
    text = re.sub(r'For media inquiries.*$', '', text, flags=re.MULTILINE)
    text = re.sub(r'\n{3,}', '\n\n', text)
    text = re.sub(r' {2,}', ' ', text)

    return text.strip()
```

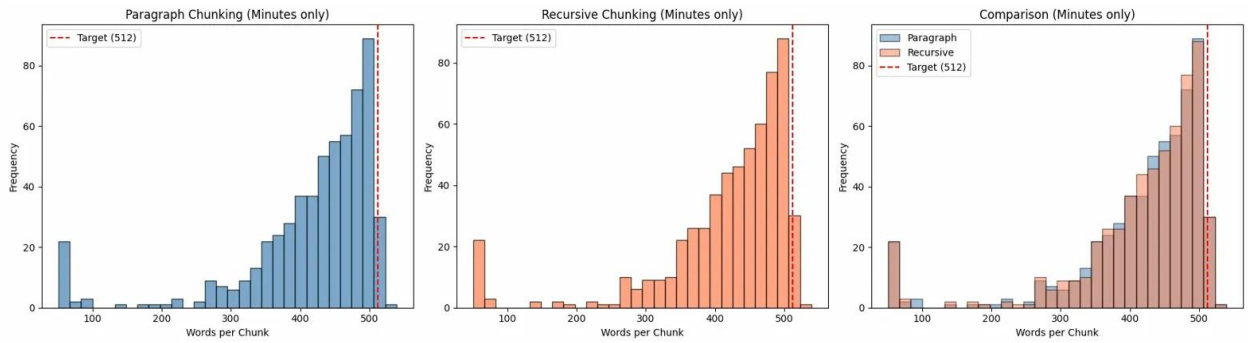
2.3 Chunking Strategy

Paragraph-aware chunking was used as it respects the natural structure of FOMC minutes. Each paragraph in the minutes typically discusses one coherent topic (e.g., inflation outlook, labour market conditions), so preserving paragraph boundaries produce more topically coherent chunks than naive fixed-size splitting.

Statements are kept as single chunks (300–500 words each) while minutes are split by merging consecutive paragraphs until reaching the target size, with sentence-level splitting as a fallback for oversized paragraphs. We generate four chunk size configurations (256, 512, 768, and 1024 words) with proportional overlap (estimated 10% of chunk size) to study the effect of chunk granularity on retrieval quality. For this pipeline, 512 was chosen as a preliminary chunk size.

Additional note: Paragraph-aware chunking was chosen over recursive splitting after empirical comparison. Both strategies were implemented and tested, producing nearly identical results: 632 vs 637 total chunks with closely similar size distributions (mean and median around 415 words and 448 words respectively with identical standard deviation). This convergence likely occurred because FOMC minutes are well-structured documents with clear paragraph breaks and naturally align with topical boundaries. Hence, recursive splitting's fallback was rarely triggered as paragraphs were already within reasonable size bounds. Thus, I ultimately decided on paragraph-aware chunking for the pipeline.

The figure below illustrates very similar distributions of paragraph and recursive chunking:

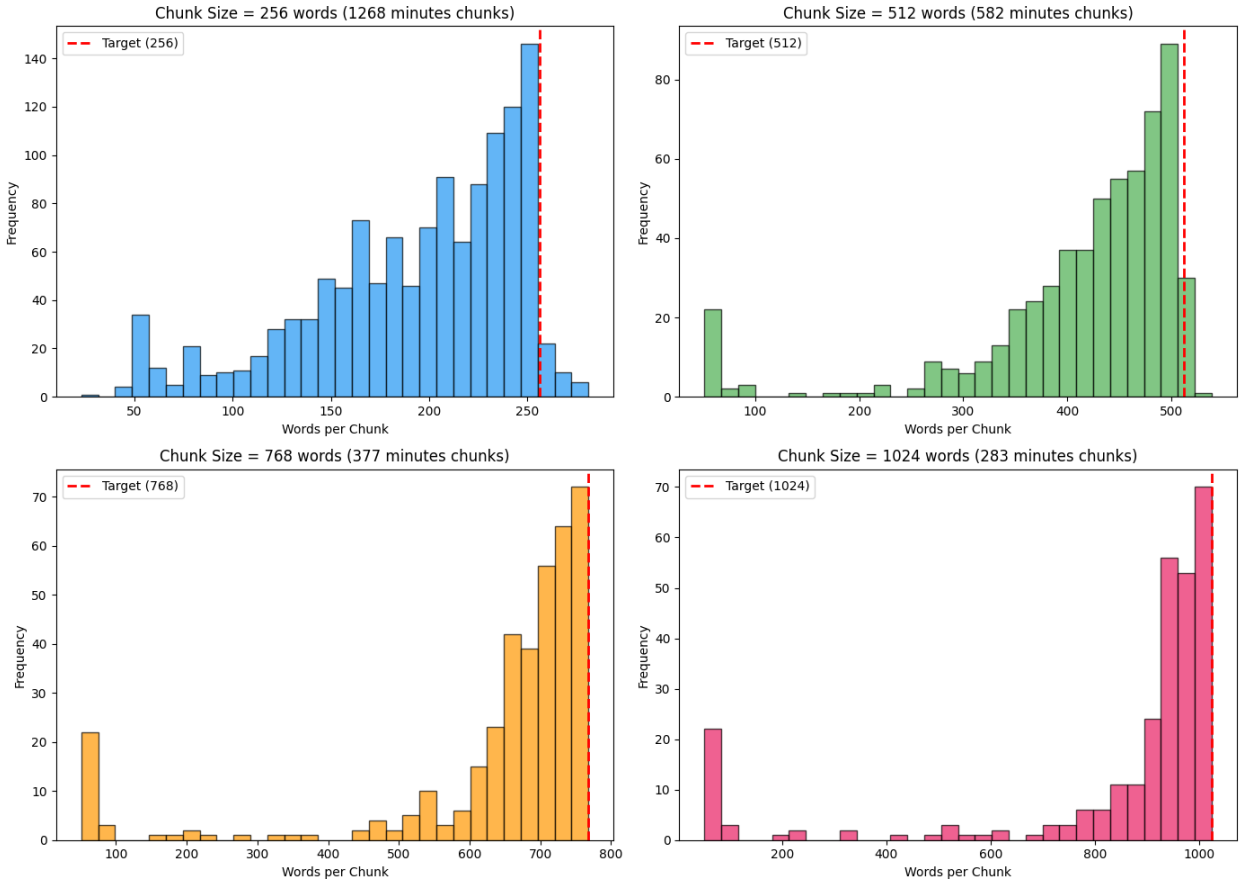


Below shows the code snippet for chunking of minutes and documents using paragraph-aware chunking:

```
def _chunk_minutes_paragraph(text, chunk_size=512, chunk_overlap=50):
    """
    Paragraph-aware chunking for FOMC minutes.
    Respects natural paragraph boundaries where possible.
    """
    paragraphs = [p.strip() for p in text.split('\n\n') if p.strip()]

    chunks = []
    current_chunk_parts = []
    current_word_count = 0

    for para in paragraphs:
        para_word_count = len(para.split())

        if para_word_count > chunk_size:
            if current_chunk_parts:
                chunks.append('\n\n'.join(current_chunk_parts))
                current_chunk_parts = []
                current_word_count = 0
            sub_chunks = _split_large_paragraph(para, chunk_size,
chunk_overlap)
            chunks.extend(sub_chunks)

        elif current_word_count + para_word_count > chunk_size:
            chunks.append('\n\n'.join(current_chunk_parts))

            if chunk_overlap > 0 and current_chunk_parts:
                overlap_text = _get_overlap_text(current_chunk_parts[-1],
chunk_overlap)
                current_chunk_parts = [overlap_text, para]
                current_word_count = len(overlap_text.split()) +
para_word_count
            else:
                current_chunk_parts = [para]
                current_word_count = para_word_count
        else:
            current_chunk_parts.append(para)
            current_word_count += para_word_count
```

```
    if current_chunk_parts:
        chunks.append('\n\n'.join(current_chunk_parts))

    return chunks

def chunk_documents(df, chunk_size=512, chunk_overlap=50):
    """
    Split all FOMC documents into retrievable chunks with metadata.
    Uses paragraph-aware chunking for minutes; keeps statements whole.
    """
    all_chunks = []

    for _, row in df.iterrows():
        doc_type = row['type']
        meeting_date = row['date'].strftime('%Y-%m-%d')
        doc_id = row['doc_id']
        text = row['text']

        if doc_type == 'Statement':
            chunks = [text]
        else:
            chunks = _chunk_minutes_paragraph(text, chunk_size,
chunk_overlap)

        for i, chunk_text in enumerate(chunks):
            all_chunks.append({
                'text': chunk_text,
                'doc_id': doc_id,
                'meeting_date': meeting_date,
                'doc_type': doc_type,
                'chunk_index': i,
                'total_chunks': len(chunks),
            })

    chunk_word_counts = [len(c['text'].split()) for c in all_chunks]
    print(f"Chunking (size={chunk_size}, overlap={chunk_overlap}): "
          f"{len(all_chunks)} chunks "
          f"({sum(1 for c in all_chunks if c['doc_type'] == 'Statement')}
statements, "
          f"{sum(1 for c in all_chunks if c['doc_type'] == 'Minute')}
minutes) | "
          f"words: mean={np.mean(chunk_word_counts):.0f}, "
          f"median={np.median(chunk_word_counts):.0f}, "
          f"min={np.min(chunk_word_counts)},
max={np.max(chunk_word_counts)}")

    return all_chunks
```

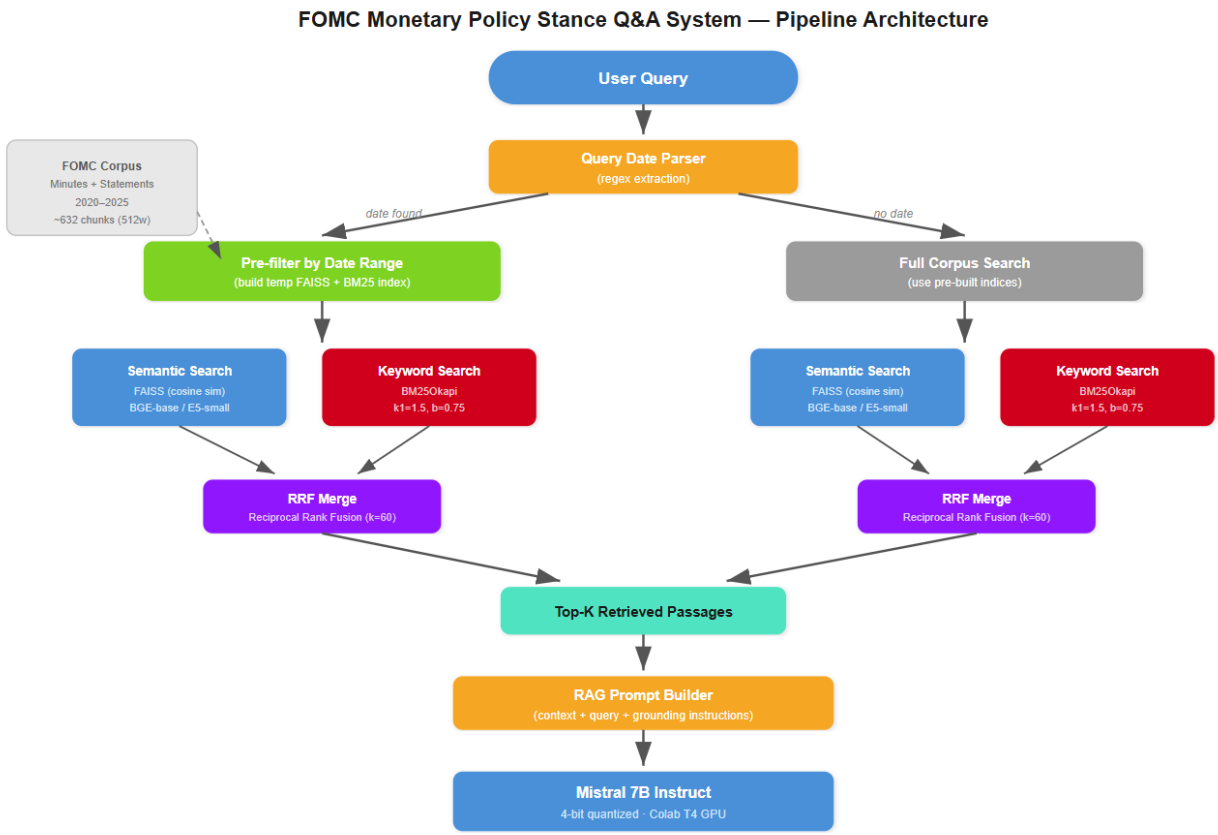
Chunking the FOMC data into various sizes we observe the distributions below. All are left-skewed with the majority of chunks clustering near but below the target size. There small spike around 50–80 words appearing consistently across all four configurations corresponds to FOMC statements, which are kept as single chunks regardless of target size. This is likely due to first or last chunks of minutes documents.


Chunk Size = 256 words (1268 minutes chunks)
Target (256)
Frequency
Words per Chunk
Chunk Size = 512 words (582 minutes chunks)
Target (512)
Frequency
Words per Chunk
Chunk Size = 768 words (377 minutes chunks)
Target (768)
Frequency
Words per Chunk
Chunk Size = 1024 words (283 minutes chunks)
Target (1024)
Frequency
Words per Chunk

3. Methodology & Implementation

3.1 Pipeline Architecture

The system implements a hybrid retrieval-augmented generation pipeline with the components described in the diagram. The pipeline processes user queries through a date parser that determines the retrieval strategy. Date-specific queries trigger pre-filtering to narrow the search space before retrieval, while general queries search the full corpus. Both paths combine semantic and keyword search via Reciprocal Rank Fusion before passing retrieved passages to Mistral 7B for grounded answer generation.



3.2 Embedding & Indexing

For embedding, two embedding models were chosen to evaluate how model quality affects retrieval:

Model	Parameters	Dimensions	Key Characteristics
intfloat/e5-small-v2	33M	384	Lightweight, fast inference, strong top-5 retrieval accuracy
BAAI/bge-base-en-v1.5	109M	768	Retrieval-optimized with hard negative mining, better at distinguishing similar passages

BGE-base was selected for its hard-negative-mining training, which might potentially help distinguish between semantically similar FOMC passages. E5-small served as a lightweight baseline to determine if pipeline design could compensate for a smaller model. Both models require query prefixes for optimal performance:

- E5 uses "*query:* " for queries and "*passage:* " for documents,
- BGE uses "*Represent this sentence for searching relevant passages:* " for queries.

For the RAG pipeline construction, BGE-base was selected as I anticipated that it may perform better, given its hard-negative-mining training.

As for indexing, FAISS was chosen for simplicity and zero-cost deployment as it better fits the current scale. Production systems with larger corpora would most likely benefit from managed vector databases with native metadata filtering such as Qdrant or Weaviate. From there FAISS IndexFlatIP indices (exact inner product search with normalized vectors, equivalent to cosine similarity) was built for all 8 configurations (2 models × 4 chunk sizes).

3.3 Hybrid Retrieval

Pure semantic search struggled with temporal specificity on FOMC documents. In an early-stage testing, a query about "inflation in March 2023" retrieved passages from 2024 and 2021 because when discussing recurring topics, FOMC minutes use highly similar language across meetings. To address this, a hybrid retrieval system was implemented with three components:

1. Query Date Parser: A regex-based extractor that identifies temporal references in queries (e.g., "March 2023" → 2023-03-01 to 2023-03-31, "early 2024" → 2024-01-01 to 2024-06-30).

2. Pre-filtering: When a date reference is detected, chunks are filtered to the relevant date range before search. We build a temporary FAISS index from only the date-filtered chunks, ensuring semantic search ranks passages within the correct time period rather than across the entire corpus. This was a critical optimization. Without pre-filtering, relevant early-2024 chunks ranked at position 182+ out of 632, completely outside any reasonable candidate pool.

3. Dual search with RRF fusion: Both semantic search (FAISS) and keyword search (BM25 with default parameters k1=1.5, b=0.75 from the original BM25 paper) were run over the same chunk set. Results are merged using Reciprocal Rank Fusion (RRF), which combines rankings without requiring score normalization across different systems. Each result is tagged with its source (semantic, BM25, or both), enabling analysis of each component's contribution.

BM25 was selected as the keyword search component using default parameters (k1=1.5, b=0.75), which are standard values from the original BM25 paper. Tuning these on a domain-specific relevance dataset could improve performance but is left for future work.

Below is the (rather lengthy) code snippet for the final hybrid retrieval function:

```
def hybrid_retrieve(query, faiss_index, bm25_index, chunks,
                    embedding_model, model_key,
                    top_k=5, semantic_top_n=20, bm25_top_n=20,
                    rrf_k=60, use_date_filter=True):
    """
    Hybrid retrieval combining semantic search, BM25, and metadata filtering.
```

```
    When a date reference is detected in the query:
        - PRE-FILTER: Narrow chunks to the date range FIRST
        - Build a temporary FAISS index from only those chunks
        - Run semantic + BM25 search on the filtered subset

    When no date reference:
        - Search the full corpus with both methods
        - Merge with RRF

    This avoids the problem of relevant-but-temporally-specific chunks
    being buried in a sea of semantically similar chunks from other periods.

    Args:
        query: Natural language query
        faiss_index: FAISS index for semantic search (used when no date
filter)
        bm25_index: BM25Okapi index for keyword search (used when no date
filter)
        chunks: List of chunk dicts
        embedding_model: Loaded SentenceTransformer
        model_key: Key in MODEL_CONFIGS
        top_k: Final number of results to return
        semantic_top_n: Candidates from semantic search (when no date filter)
        bm25_top_n: Candidates from BM25 search (when no date filter)
        rrf_k: RRF constant (default 60)
        use_date_filter: Whether to apply date-based filtering

    Returns:
        results: List of top-k result dicts, sorted by RRF score
        metadata: Dict with retrieval diagnostics
    """
    # Step 1: Parse date from query
    start_date, end_date = parse_date_from_query(query) if use_date_filter
else (None, None)

    # Step 2: Determine search strategy
    if start_date and end_date:
        # --- PRE-FILTER STRATEGY ---
        # Narrow to date-relevant chunks first, then search within that
subset

        # Filter chunks by date
        date_filtered_indices = [
            i for i, c in enumerate(chunks)
            if start_date <= c['meeting_date'] <= end_date
        ]
        filtered_chunks = [chunks[i] for i in date_filtered_indices]

        if len(filtered_chunks) == 0:
            # No chunks in date range — fall back to full corpus search
            semantic_results = query_index(
                query, faiss_index, chunks, embedding_model, model_key,
                top_k=semantic_top_n
            )
            for r in semantic_results:
                r['source'] = 'semantic'
```

```
            bm25_results = bm25_search(query, bm25_index, chunks,
top_k=bm25_top_n)
            semantic_filtered = semantic_results
            bm25_filtered = bm25_results
            filter_note = f"{start_date} to {end_date} (no chunks found, fell
back to full corpus)"
        else:
            # Build temporary FAISS index from filtered chunks
            config = MODEL_CONFIGS[model_key]
            passage_prefix = config['passage_prefix']

            filtered_texts = [passage_prefix + c['text'] for c in
filtered_chunks]
            filtered_embeddings = embedding_model.encode(
                filtered_texts, normalize_embeddings=True
            ).astype('float32')

            temp_index = faiss.IndexFlatIP(filtered_embeddings.shape[1])
            temp_index.add(filtered_embeddings)

            # Semantic search on filtered subset
            query_prefix = config['query_prefix']
            query_embedding = embedding_model.encode(
                [query_prefix + query], normalize_embeddings=True
            ).astype('float32')

            sem_k = min(semantic_top_n, len(filtered_chunks))
            scores, indices = temp_index.search(query_embedding, sem_k)

            semantic_filtered = []
            for rank, (score, idx) in enumerate(zip(scores[0], indices[0])):
                chunk = filtered_chunks[idx]
                semantic_filtered.append({
                    'rank': rank + 1,
                    'score': float(score),
                    'meeting_date': chunk['meeting_date'],
                    'doc_type': chunk['doc_type'],
                    'chunk_index': chunk['chunk_index'],
                    'text': chunk['text'],
                    'text_preview': chunk['text'][:200],
                    'source': 'semantic',
                })

            # BM25 search on filtered subset
            filtered_tokenized = [c['text'].lower().split() for c in
filtered_chunks]
            temp_bm25 = BM25Okapi(filtered_tokenized)
            query_tokens = query.lower().split()
            bm25_scores = temp_bm25.get_scores(query_tokens)

            bm25_k = min(bm25_top_n, len(filtered_chunks))
            top_bm25_indices = np.argsort(bm25_scores)[::-1][:bm25_k]

            bm25_filtered = []
            for rank, idx in enumerate(top_bm25_indices):
                chunk = filtered_chunks[idx]
                bm25_filtered.append({
```

```
                    'rank': rank + 1,
                    'score': float(bm25_scores[idx]),
                    'meeting_date': chunk['meeting_date'],
                    'doc_type': chunk['doc_type'],
                    'chunk_index': chunk['chunk_index'],
                    'text': chunk['text'],
                    'text_preview': chunk['text'][:200],
                    'source': 'bm25',
                })

            filter_note = f"{start_date} to {end_date} (pre-filtered to
{len(filtered_chunks)} chunks)"
    else:
        # --- NO DATE FILTER: search full corpus ---
        semantic_results = query_index(
            query, faiss_index, chunks, embedding_model, model_key,
            top_k=semantic_top_n
        )
        for r in semantic_results:
            r['source'] = 'semantic'

        bm25_results = bm25_search(query, bm25_index, chunks,
top_k=bm25_top_n)

        semantic_filtered = semantic_results
        bm25_filtered = bm25_results
        filter_note = "None"

    # Step 3: Reciprocal Rank Fusion
    rrf_scores = {}
    chunk_lookup = {}

    for rank_idx, r in enumerate(semantic_filtered):
        key = (r['meeting_date'], r['chunk_index'], r['doc_type'])
        rrf_score = 1.0 / (rrf_k + rank_idx + 1)
        rrf_scores[key] = rrf_scores.get(key, 0) + rrf_score
        if key not in chunk_lookup:
            chunk_lookup[key] = r

    for rank_idx, r in enumerate(bm25_filtered):
        key = (r['meeting_date'], r['chunk_index'], r['doc_type'])
        rrf_score = 1.0 / (rrf_k + rank_idx + 1)
        rrf_scores[key] = rrf_scores.get(key, 0) + rrf_score
        if key not in chunk_lookup:
            chunk_lookup[key] = r

    # Step 4: Sort by RRF score and return top-k
    sorted_keys = sorted(rrf_scores.keys(), key=lambda k: rrf_scores[k],
reverse=True)

    final_results = []
    for rank, key in enumerate(sorted_keys[:top_k]):
        result = chunk_lookup[key].copy()
        result['rrf_score'] = rrf_scores[key]
        result['rank'] = rank + 1
        in_semantic = any(
            (r['meeting_date'], r['chunk_index'], r['doc_type']) == key
```

```
            for r in semantic_filtered
        )
        in_bm25 = any(
            (r['meeting_date'], r['chunk_index'], r['doc_type']) == key
            for r in bm25_filtered
        )
        result['found_by'] = []
        if in_semantic:
            result['found_by'].append('semantic')
        if in_bm25:
            result['found_by'].append('bm25')
        final_results.append(result)

    # Diagnostics
    metadata = {
        'query': query,
        'date_filter': filter_note,
        'semantic_candidates': len(semantic_filtered),
        'bm25_candidates': len(bm25_filtered),
        'unique_chunks_merged': len(rrf_scores),
        'final_top_k': len(final_results),
    }

    return final_results, metadata

print("Hybrid retrieval function defined (with pre-filtering).")
```

3.4 Generation

Finally, for the generation stage, Mistral 7B Instruct (v0.3) was chosen. With 4-bit NF4 quantization via bitsandbytes, reducing memory requirements from 14GB to about 4GB, fitting comfortably on Google Colab's free T4 GPU. The deliberate choice of a lightweight model was to support our solution that RAG can enable small open-source models to produce accurate policy analysis.

The RAG prompt instructs the model to:

(1) only state information supported by the provided context,
(2) cite meeting dates when referencing specific information,
(3) explicitly acknowledge when context is insufficient, and
(4) avoid adding information from its own knowledge.

For baseline comparison, detailed later in Evaluation & Result, a separate ‘no-RAG’ prompt asks the same question without context, allowing the model to answer from training data alone.

```
def build_rag_prompt(query, retrieved_results):
    """
    Build the RAG prompt that combines user query with retrieved context.

    The prompt uses Mistral's instruction format and instructs the model to:
    - Only use information from the provided context
    - Cite meeting dates when making claims
    - Acknowledge when context is insufficient

    Args:
        query: User's question
```

```
        retrieved_results: List of result dicts from hybrid_retrieve()

    Returns: Formatted prompt string
    """
    # Build context string from retrieved passages
    context_parts = []
    for r in retrieved_results:
        meeting_date = r['meeting_date']
        doc_type = r['doc_type']
        text = r['text']
        context_parts.append(
            f"[Source: FOMC {doc_type}, Meeting date:
{meeting_date}]\n{text}"
        )

    context_str = "\n\n---\n\n".join(context_parts)

    # Build the instruction prompt
    system_instruction = (
        "You are a Federal Reserve policy analyst. Answer the user's question
"
        "about FOMC monetary policy based ONLY on the provided context
passages. "
        "Follow these rules strictly:\n"
        "1. Only state information that is directly supported by the context
passages.\n"
        "2. Cite the meeting date when referencing specific information "
        "(e.g., 'In the March 2023 meeting...').\n"
        "3. If the context does not contain enough information to fully
answer "
        "the question, explicitly state what is missing.\n"
        "4. Do not add information from your own knowledge — only use the
provided context.\n"
        "5. Be concise and specific."
    )

    # Mistral instruction format
    prompt = f"[INST] {system_instruction}\n\n"
    prompt += f"Context passages:\n\n{context_str}\n\n"
    prompt += f"Question: {query} [/INST]"

    return prompt

def build_no_rag_prompt(query):
    """
    Build a prompt WITHOUT retrieval context (for baseline comparison).
    The model must answer from its own knowledge only.
    """
    system_instruction = (
        "You are a Federal Reserve policy analyst. Answer the following
question "
        "about FOMC monetary policy. Be specific and cite meeting dates where
possible. "
        "Be concise."
    )

    prompt = f"[INST] {system_instruction}\n\nQuestion: {query} [/INST]"
```

```
    return prompt

print("Prompt templates defined.")
```

4. Evaluation & Results

4.1 Evaluation Framework

We evaluate across three dimensions using a combination of automatic metrics and LLM-as-judge scoring (Claude Haiku 4.5 via API):

Category	Metric	Reason
Retrieval	Precision@k	Measures retrieval quality to facilitate generation.
Retrieval	Mean Reciprocal Rank (MRR)	
Generation	ROUGE-L	Provides an automatic, reproducible baseline for generation quality. *(Although it's somewhat limited as differently phrased but accurate answers might score low.)*
Generation	Faithfulness (1–5)	Ensures that the model stays grounded in context.
Generation	Relevance (1–5)	General assessment to see if the model answered the right question, and if it covered enough in its answer.
Generation	Completeness (1–5)	

It should be further noted that because our dataset is not labelled as "relevant" or "not relevant" for retrieval evaluation, we also got Claude Haiku to rate the retrieved chunks as "`relevant`", "`partially_relevant`" or "`not_relevant`". This subsequently allowed for Precision@k and MRR to be calculated.

We test on 15 queries spanning 9 topics (inflation, labour market, rate decisions, housing, tariffs, AI, etc.), 3 difficulty levels, and a mix of date-specific and general queries. Each query has a hand-written reference answer for ROUGE-L scoring.

Below is the code snippet for running ROUGE-L scoring on both RAG and non-RAG queries:

```
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=True)

for r in all_results:
    # ROUGE-L: RAG answer vs reference
    rag_rouge = scorer.score(r['reference_answer'], r['rag_answer'])
    r['rag_rouge_l'] = rag_rouge['rougeL'].fmeasure

    # ROUGE-L: Base answer vs reference
    base_rouge = scorer.score(r['reference_answer'], r['no_rag_answer'])
    r['base_rouge_l'] = base_rouge['rougeL'].fmeasure

# Summary
```

```
rag_scores = [r['rag_rouge_l'] for r in all_results]
base_scores = [r['base_rouge_l'] for r in all_results]

print("ROUGE-L Scores (vs reference answers)")
print("=" * 50)
print(f"  RAG:  mean={np.mean(rag_scores):.3f},
median={np.median(rag_scores):.3f}")
print(f"  Base: mean={np.mean(base_scores):.3f},
median={np.median(base_scores):.3f}")
print(f"\n  Per query:")
for r in all_results:
    print(f"    {r['query'][:50]:50s}
RAG={r['rag_rouge_l']:.3f}  Base={r['base_rouge_l']:.3f}")
```

This is the code snippet for calling prompting Claude Haiku 4.5 as a judge and running Faithfulness, Relevance and Completeness evaluations:

```
def judge_answer(query, answer, retrieved_chunks, metric):
    """
    Use Claude Haiku to score a generated answer.
    Returns integer score 1-5.
    """
    context_str = ""
    if retrieved_chunks:
        context_parts = []
        for r in retrieved_chunks:
            context_parts.append(f"[{r['meeting_date']}, {r['doc_type']}]:
{r['text'][:500]}")
        context_str = "\n\n".join(context_parts)

    prompts = {
        "faithfulness": f"""Rate the faithfulness of this answer on a scale
of 1-5.
A faithful answer ONLY contains information supported by the provided
context.

Scoring:
5 = Every claim is directly supported by the context
4 = Almost all claims supported, minor unsupported details
3 = Most claims supported but some notable unsupported additions
2 = Significant unsupported claims mixed with supported ones
1 = Mostly unsupported or hallucinated information

Context passages:
{context_str}

Question: {query}
Answer: {answer}

Reply with ONLY a single number (1-5):""",

        "relevance": f"""Rate how relevant this answer is to the question on
a scale of 1-5.

Scoring:
5 = Directly and completely addresses the question
```

```
4 = Mostly addresses the question with minor gaps
3 = Partially addresses the question
2 = Tangentially related but doesn't answer the question
1 = Completely off-topic

Question: {query}
Answer: {answer}

Reply with ONLY a single number (1-5):""",

        "completeness": f"""Rate the completeness of this answer on a scale
of 1-5.
A complete answer covers the key points relevant to the question.

Scoring:
5 = Covers all major points comprehensively
4 = Covers most key points with minor omissions
3 = Covers some key points but misses important information
2 = Only covers a small portion of relevant information
1 = Misses nearly all key information

Question: {query}
Answer: {answer}

Reply with ONLY a single number (1-5):"""
    }

    response = call_claude_judge(prompts[metric])

    if response:
        # Extract the number from response
        import re
        numbers = re.findall(r'[1-5]', response)
        if numbers:
            return int(numbers[0])

    return None

print("Judge function defined.")


# Run LLM judge on all answers
metrics = ['faithfulness', 'relevance', 'completeness']

print("Running Claude Haiku evaluation...")
print("=" * 70)

for i, r in enumerate(all_results):
    print(f"\n[{i+1}/{len(all_results)}] {r['query'][:50]}...")

    for metric in metrics:
        # Score RAG answer
        rag_score = judge_answer(
            r['query'], r['rag_answer'], r['retrieved_passages'], metric
        )
        r[f'rag_{metric}'] = rag_score
```

```
        # Score base answer (no context for faithfulness — judge on general
accuracy)
        base_score = judge_answer(
            r['query'], r['no_rag_answer'], [], metric
        )
        r[f'base_{metric}'] = base_score

        print(f"  {metric}: RAG={rag_score}, Base={base_score}")

        # Rate limiting pause
        time.sleep(1)

print("\n" + "=" * 70)
print("LLM judge evaluation complete!")
```

Finally here is the code snippet prompting Claude Haiku 4.5 as a judge to evaluate if chucks are "`relevant`", "`partially_relevant`" or "`not_relevant`", before proceeding to compute the Precision@k and MRR:

```
def judge_retrieval_relevance(query, chunk_text, meeting_date, doc_type):
    """
    Use Claude Haiku to judge if a retrieved chunk is relevant to the query.
    Returns: 'relevant', 'partially_relevant', or 'not_relevant'
    """
    prompt = f"""Given the following query and retrieved passage, rate the
relevance.

- RELEVANT: The passage directly answers or informs the query
- PARTIALLY RELEVANT: The passage is related but doesn't directly answer
- NOT RELEVANT: The passage is unrelated to the query

Query: {query}
Passage (from FOMC {doc_type}, {meeting_date}): {chunk_text[:800]}

Reply with ONLY one of: RELEVANT, PARTIALLY RELEVANT, NOT RELEVANT"""

    response = call_claude_judge(prompt)

    if response:
        response_lower = response.lower().strip()
        if 'not relevant' in response_lower or 'not_relevant' in
response_lower:
            return 'not_relevant'
        elif 'partially' in response_lower:
            return 'partially_relevant'
        elif 'relevant' in response_lower:
            return 'relevant'

    return None

# Evaluate retrieval for all queries
print("Evaluating retrieval quality...")
print("=" * 70)

for i, r in enumerate(all_results):
```

```
    print(f"\n[{i+1}/{len(all_results)}] {r['query'][:50]}...")

    relevance_judgments = []

    for passage in r['retrieved_passages']:
        judgment = judge_retrieval_relevance(
            r['query'], passage['text'], passage['meeting_date'],
passage['doc_type']
        )
        relevance_judgments.append(judgment)
        time.sleep(0.5)  # Rate limiting

    r['relevance_judgments'] = relevance_judgments

    # Compute Precision@k
    if relevance_judgments:
        relevant_count = sum(
            1 for j in relevance_judgments if j == 'relevant'
        ) + 0.5 * sum(
            1 for j in relevance_judgments if j == 'partially_relevant'
        )
        r['precision_at_k'] = relevant_count / len(relevance_judgments)
    else:
        r['precision_at_k'] = 0.0

    # Compute MRR
    r['mrr'] = 0.0
    for rank, j in enumerate(relevance_judgments):
        if j in ('relevant', 'partially_relevant'):
            r['mrr'] = 1.0 / (rank + 1)
            break

    print(f"  Judgments: {relevance_judgments}")
    print(f"  Precision@{len(relevance_judgments)}:
{r['precision_at_k']:.2f}, MRR: {r['mrr']:.2f}")

print("\n" + "=" * 70)
print("Retrieval evaluation complete!")
```

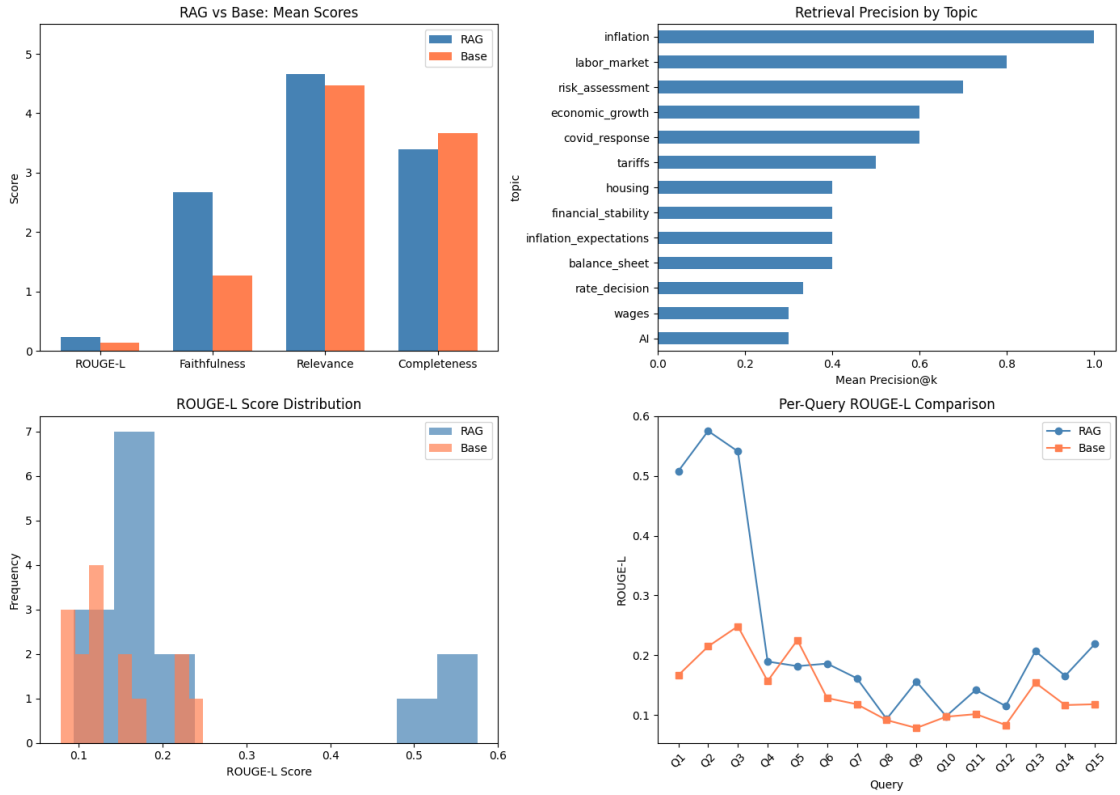
4.2 RAG vs Base Model Comparison

Metric	RAG	Base	Δ
ROUGE-L	0.236	0.140	+0.096 (+69%)
Faithfulness	2.667	1.267	+1.400 (+111%)
Relevance	4.667	4.467	+0.200
Completeness	3.400	3.667	−0.267

RAG's strongest improvement is in faithfulness (+111%), confirming that retrieval grounds the model's outputs in actual documents. The base model scored near 1 (mostly hallucinated) on almost every query. ROUGE-L improved by 69%, with the largest gains on date-specific queries. Completeness slightly favours the base model (−0.267), due to its likely tendency to

generate longer, more elaborate answers, although these complete answers often contain unverifiable claims.

Below, the figures evaluate a more detailed performance of RAG vs the base model alone. There, we see how the best topic for retrieval is inflation, possible due to its more consistent terminology which makes it easier to search for. But overall, we can observe that RAG improves the model performances.



4.3 Retrieval Performance

Across all queries, the aggregated mean Precision@k across all queries was 0.493, with mean MRR of 0.883. As our pipeline divides between queries with dates and those without, I've further broken down these values accordingly in the table below. Sure enough, performance varied substantially based on whether queries contained date references:

Query Type	Mean Precision@k	Mean MRR
With date reference (n=6)	0.667	1.000
Without date reference (n=9)	0.378	0.806
Average of both above	0.493	0.883

Date-specific queries achieve perfect MRR (1.0), meaning the pre-filtering mechanism consistently surfaces relevant documents at rank 1. Queries without date references show notably lower precision, reflecting the challenge of differentiating semantically similar passages across multiple years of FOMC discussions.

4.4 Retrieval Ablation Study

A brief, single-query ablation study was conducted to analyse the contribution of components to this relatively elaborate pipeline. Using the query "*What was the Fed's stance on inflation in March 2023?*" the following three configurations were compared:

1. **Semantic only (no filter, no BM25):** Top results were from December 2024, September 2024, and January 2025. None from the target period. This shows how pure embedding similarity cannot distinguish between meetings discussing inflation in similar language.
2. **Semantic + date filter:** Successfully narrowed to March 2023 documents, but relevant chunks were ranked 13th and 46th in the unfiltered semantic ranking. This demonstrated that pre-filtering (not post-filtering) is essential to the retrieval process.
3. **Full hybrid (semantic + BM25 + date filter):** The March 2023 minutes chunk was boosted to rank 1 with a higher RRF score, because both semantic and BM25 search found it. Consequently, the BM25 contributed in boosting the rank through RRF fusion rather than finding new documents and adding extra noise.

4.5 Chunk Size Comparison

As earlier mentioned in section 2.3, a chunk size comparison is done to further investigate if the 512-chunk size truly captured the appropriate context window for retrieval.

Chunk Size	Average Retrieved	Mean Precision@k	Mean MRR
256 words	4.6	0.625	0.917
512 words	4.6	0.550	0.844
768 words	4.6	0.588	1.000
1024 words	4.6	0.562	0.906

The table shows how chunk size has marginal impact on retrieval quality for this corpus. The 256-word configuration achieved the highest precision (0.625) while 768 achieved perfect MRR (1.000). The small differences suggest that the chunk size parameter has minimal effect on the retrieval performance. This is likely because of FOMC minutes' natural paragraph structure being more consistent, comprising of clear topical boundaries.

4.6 Embedding Model Comparison

As earlier mentioned in section 3.2, the E5-small model was chosen to: 1. benchmark the BGE-base model and 2. determine if our embedding model can be replaced with a smaller one, minimising computing cost.

Model	Average Retrieved	Mean Precision@k	Mean MRR
E5-small (33M)	4.6	0.538	0.838
BGE-base (109M)	4.6	0.588	0.844

BGE-base marginally outperforms E5-small on both metrics, but the gap is smaller than expected given BGE-base is 3 times larger. This suggests that the other components such as the pre-filtering and BM25 components, are the bigger contributors to the hybrid retrieval pipeline, further confirming the ablation study in section 4.4. Hence, it is therefore possible to replace our BGE-base embedding model with E5-small and reduce the need for larger, more expensive models.

4.7 Sample Output

Here are 2 of the sample queries and the outputs generated:

Sample 1: RAG preventing a hallucination

Query: "What was the Fed's interest rate decision in September 2024?"

RAG Answer: "The Fed's interest rate decision in September 2024 was to lower the target range for the federal funds rate to 4-3/4 to 5 percent. This decision was made in the September 18, 2024 meeting."

Reference Material: "In September 2024, the FOMC decided to lower the target range for the federal funds rate by 1/2 percentage point to 4-3/4 to 5 percent, reflecting greater confidence that inflation was moving sustainably toward 2 percent and a shift in the balance of risks."

Base Answer (without RAG): "In September 2024, the Federal Open Market Committee (FOMC) did not announce any changes to its federal funds target range at their meeting on September 16-17, 2024. The target range remained at 3.50% - 3.75%, which had been set at the previous meeting in July 2024. This decision reflected the committee's ongoing efforts to balance inflation concerns with maintaining economic growth. However, it is important to note that actual decisions can vary based on the specific circumstances and data available at each meeting, so this information may change if new data emerges or unforeseen events occur. For more detailed information, please refer to the official FOMC meeting minutes or statements released after each meeting."

Comments: RAG correctly states: "The Fed's interest rate decision in September 2024 was to lower the target range for the federal funds rate to 4-3/4 to 5 percent." The base model, however, confidently states the opposite: "The FOMC did not announce any changes...The target range remained at 3.50% - 3.75%." It fabricated a completely wrong rate, a wrong meeting date ("September 16-17" instead of September 18), and claimed no change was made when in fact September 2024 was the first rate cut in the cycle — a historically significant 50 basis point reduction. In a financial context, acting on the base model's answer could lead to materially wrong investment decisions.

Sample 2: Completeness trade-off

Query: "How did the FOMC describe the labor market in early 2024?"

RAG Answer: "In early 2024, according to the FOMC, job gains have moderated since early last year but remain strong, and the unemployment rate has remained low. [Source: FOMC Statement, Meeting date: 2024-01-31]"

Reference Material: “In early 2024, the FOMC described job gains as having moderated since early last year but remaining strong, with the unemployment rate remaining low. The Committee judged that risks to achieving employment and inflation goals were moving into better balance.

Base Answer (without RAG): “In early 2024, according to the Federal Open Market Committee (FOMC), the labor market continued to strengthen. The FOMC's statement from the January 31-February 1, 2024 meeting noted that "job gains remained robust" and "the unemployment rate declined further." However, they also acknowledged that "slack remained in some labor markets, especially among lower-wage workers." This assessment was reiterated in the March 28-29, 2024 meeting as well, with the FOMC stating that "the labor market had continued to tighten, with strong job growth and a declining unemployment rate.”

Comments: The RAG answer is concise and accurate: "job gains have moderated since early last year but remain strong, and the unemployment rate has remained low." This is drawn directly from the January 2024 statement (faithfulness = 5). The base model produces a longer, more detailed answer citing specific quotes, but these quotes are fabricated. The base claims the FOMC said "slack remained in some labor markets, especially among lower-wage workers" and references a "March 28-29, 2024 meeting", but no such meeting exists. The actual date was March 19-20. The base model also states the unemployment rate "declined further" when the actual FOMC language was that it "remained low."

5. Discussion & Critical Analysis

5.1 Where the System Succeeds

Date-specific queries: The combination of date parsing and pre-filtering produces great results. Precision@k of 0.667 and 1.000 MRR for date-specific queries demonstrates that when temporal context is available, the pipeline reliably surfaces the right documents.

Faithfulness: RAG more than doubles faithfulness scores compared to the base model. Every RAG answer traces to specific source documents, providing the audit trail that financial applications require.

Lightweight model viability: Mistral 7B with 4-bit quantization, on a free Colab T4 GPU, producing grounded policy analysis when augmented with retrieval.

5.2 Where the System Struggles

Temporal ambiguity: Queries without explicit date references (e.g., "What reasons did the Fed give for pausing rate hikes?") achieve only 0.378 Precision@k. The system cannot infer that "rate pause" implicitly refers to mid-2023, a fundamental limitation of embedding-based retrieval.

Instruction following: Despite explicit prompting to stay grounded, Mistral 7B occasionally adds unsupported claims, resulting in RAG faithfulness scores averaging 2.67 out of 5. Larger instruction-tuned models would likely improve this, but at the cost of the lightweight deployment.

Completeness trade-off: RAG answers are more concise than base answers (0.267 lower in completeness), as it is constrained by the retrieved context. When retrieval returns only 2

passages (e.g., for narrow date queries), the generated answer is necessarily limited. Demonstrating the trade-off of verifiability over comprehensiveness.

5.3 Challenges Encountered

Post-filtering failure: Our initial approach of retrieving top-100 candidates from the full corpus and then filtering by date produced zero results for some queries. After doing some investigating, it was revealed that relevant chunks ranked at position 182+ in the global semantic ranking because repetitive language of FOMC minutes makes it impossible for embedding similarity alone to prioritize the correct time period. The solution was pre-filtering: narrowing the search space before applying semantic search.

LLM judge non-determinism: Running the same evaluation twice produced different scores, particularly on borderline cases (e.g., a passage judged "partially relevant" in one run and "not relevant" in another). It should be noted that evaluation results should be interpreted with appropriate uncertainty margins.

BM25 limitations on fuzzy temporal queries: BM25 matches exact tokens and cannot resolve fuzzy references like "early 2023" to specific meeting dates. For the query about "early 2023", BM25 returned zero results after date filtering because the exact phrase doesn't appear in FOMC text. Adding the semantic search helped to compensate for these cases.

5.4 Future Work

RoBERTa stance enrichment: Integrating a fine-tuned hawkish/dovish classifier (from assignment 1) to label retrieved passages with policy stance signals would add a quantified dimension to the qualitative answers.

Replacing embedding model BGE-base with E5-small: As discovered in section 4.6, swapping to an embedding model 3 times smaller has minimal impact on retrieval performance, but saves on computing resources.

Additional metadata filtering: Extracting structured other metadata from documents like policy actions (hike/cut/hold), Fed funds rate at time of meeting, vote splits will enable richer query filtering. Document type filtering (statements vs minutes) might also improve precision for specific query types.

Advanced date handling: Replacing regex-based date parsing with a library like dateparser for broader coverage of natural language date expressions and using an LLM to extract temporal intent from ambiguous queries.

BM25 parameter tuning: The current implementation uses default BM25 parameters (k1=1.5, b=0.75). Tuning these on a domain-specific relevance dataset could improve keyword retrieval quality.

Production scaling: The current FAISS flat index with pre-filtering works well at our corpus scale (~600 chunks) but would not scale to millions of documents. A production system would benefit from vector databases like Qdrant, Weaviate that have native metadata and approximate nearest neighbour indices.

Multiple judge runs: Averaging LLM judge scores across multiple evaluation runs would reduce non-determinism and produce more reliable metrics, analogous to using multiple human annotators.

6. Conclusion

Overall, this project demonstrates that a well-designed RAG pipeline can enable a lightweight open-source model (Mistral 7B, 4-bit quantized) to produce grounded, verifiable answers about Federal Reserve monetary policy. The system achieves improvement in faithfulness and in ROUGE-L over the base model, with the strongest gains on date-specific queries where retrieval precision reaches 1.0 and MRR is perfect.

The hybrid retrieval approach (semantic search + BM25 keyword matching, and date-based pre-filtering with Reciprocal Rank Fusion) proved essential for handling the temporal specificity in FOMC documents. The date parser and pre-filtering mechanism were the most impactful pipeline optimizations, transforming retrieval from ineffective (relevant passages ranked 182nd) to precise (relevant passages ranked 1st).

However, the system has clear limitations. Queries without explicit temporal references remain challenging, with retrieval precision dropping from 0.667 to 0.378. The base model occasionally matches or exceeds RAG on completeness, reflecting RAG's inherent trade-off between verifiability and comprehensiveness. Additionally, Mistral 7B's instruction-following is imperfect. Even with grounding instructions, the model sometimes adds unsupported claims.

These findings suggest that RAG's primary value in financial NLP is not simply improving answer quality but fundamentally changing the nature of the output from plausible-but-unverifiable to grounded-and-traceable. In domains where a confidently wrong answer is more dangerous than an incomplete one, this shift is significant. Future work could enhance the system through additional metadata filtering, fine-tuned domain-specific embeddings, and integration of stance classification models to enrich answers with quantified policy signals.